

MÔ HÌNH TÍCH HỢP LIÊN TỤC

Hệ thống Quản lý và Số hóa Tài liệu Thư viện HCMUS (HCMUS-LDMS)

Thông tin tài liệu

Trường	Nội dung
Mã tài liệu	HCMUS-LDMS-CI
Chủ sở hữu	DevOps — Nguyễn Tuấn Anh; Nguyễn Quang Thái
Phiên bản	1.0 — 22/08/2026
Nguồn cấu hình	.github/workflows/ci.yml
Trạng thái	Mô hình phản ánh cấu hình repository; kết quả chạy/email cần evidence GitHub thực tế

Mục lục

- 1. Mục đích
- 2. Mô hình hiện tại
- 3. Các bước kiểm tra
- 4. Kích hoạt và quy tắc hợp nhất
- 5. Thông báo
- 6. Bảo mật và xử lý lỗi
- 7. Evidence và điểm chưa thống nhất

1. Mục đích

Tích hợp liên tục giúp kiểm tra mỗi thay đổi trước hoặc ngay khi tích hợp, phát hiện sớm lỗi định dạng, mã nguồn, kiểm thử và build. Cấu hình dùng GitHub Actions, chạy backend và frontend độc lập, sau đó gửi thông báo cho lần đẩy lên `main`.

2. Mô hình hiện tại

Thành viên → Git/GitHub → GitHub Actions → Backend checks + Frontend checks → Tổng hợp trạng thái → Thông báo email khi đẩy main.

Thành phần	Công cụ	Vai trò
Quản lý phiên bản	Git/GitHub	Lưu thay đổi và Pull Request
Máy chủ CI	GitHub Actions	Điều phối jobs trên Ubuntu
Backend	uv, Ruff, Pytest	Cài phụ thuộc, kiểm tra định dạng/lint và test
Frontend	Node.js 20, npm	Cài phụ thuộc, lint, build và test
Thông báo	Python script + Brevo	Gửi kết quả backend/frontend sau push main

3. Các bước kiểm tra

3.1. Backend

1. Checkout mã nguồn.
2. Cài `uv`.
3. `uv sync`.
4. `uv run ruff format --check .`.
5. `uv run ruff check .`.
6. `uv run pytest`.

3.2. Frontend

1. Checkout mã nguồn.
2. Cài Node.js 20.
3. `npm ci`.
4. `npm run lint`.
5. `npm run build`.
6. `npm test`.

Một job thất bại phải được xem tại log cụ thể. Tài liệu không khẳng định các lệnh đang đặt vì chưa có run ID/report được đính kèm trong `docs.1`.

4. Kích hoạt và quy tắc hợp nhất

Cấu hình hiện tại chạy Pull Request đến `main` hoặc `develop`, và push lên `main`, `develop`, `release/**`, `hotfix/**`. Baseline tài liệu dùng Trunk-Based với `main` và nhánh task/fix ngắn; các trigger `develop/release/hotfix` là phần tương thích còn trong workflow, không chứng minh nhóm đang dùng GitFlow.

Quy tắc đề xuất:

- Pull Request liên quan story phải ghi Story ID và tác động.
- Backend/frontend check áp dụng theo vùng thay đổi nhưng gate không được bỏ qua nếu có phụ thuộc chung.
- Chỉ hợp nhất khi checks bắt buộc đạt, reviewer đồng ý và không có secret.
- Nhánh task tồn tại ngắn; sau hợp nhất cập nhật board/evidence.
- Branch protection thực tế phải được chụp/xuất bằng chứng từ GitHub; chưa có evidence trong bộ này.

5. Thông báo

Job `notify` chạy sau backend/frontend đối với push lên `main`, dù job trước đạt hay lỗi. Nó dùng biến bí mật `BREVO_API_KEY`, `BREVO_SENDER_EMAIL` và script `app.scripts.send_merge_notification`. Danh sách người nhận nằm ở `.github/ci-notify-recipients.txt`.

`continue-on-error: true` nghĩa lỗi gửi email không làm thay đổi kết quả kiểm tra sản phẩm. Vì vậy email chỉ là thông báo, không phải quality gate. Bản in email phải lấy từ lần chạy thật và che dữ liệu nhạy cảm.

6. Bảo mật và xử lý lỗi

- Secrets lưu trong GitHub Secrets, không ghi giá trị vào workflow/tài liệu.
- Pull Request từ nguồn không tin cậy không được cấp secrets nhạy cảm ngoài cơ chế GitHub.
- Dependency/script cài đặt cần được review; action nên được cập nhật có kiểm soát.
- Khi job lỗi: mở log, xác định bước/commit, sửa trên nhánh, chạy lại và liên kết run vào Evidence Index.
- Không dùng cách bỏ test hoặc sửa expected result không căn cứ để làm pipeline xanh.

7. Evidence và điểm chưa thống nhất

Evidence cần nộp	Trạng thái
Bản in <code>.github/workflows/ci.yml</code>	Có nguồn trong repository
GitHub Actions run cho backend/frontend	Chưa liên kết trong tài liệu
Pull Request có checks và review	Chưa liên kết trong tài liệu
Email thông báo kết quả	Chưa liên kết trong tài liệu
Branch protection/rule set	Chưa xác minh

Cần quyết định có bỏ trigger `develop/release/hotfix` để khớp hoàn toàn Trunk-Based hay giữ cho tương thích. Mọi thay đổi workflow phải qua review và cập nhật [Process](#), [Quality Plan](#) và [ADR](#).